

Advanced Natural Language Processing Pre-Training and Fine- Tuning Large-Language Models

Department of Advanced Computer Sciences (DACS)
Master of AI / DSDM
Prof dr ir Jan C. Scholtes and Dr. Aki Harma – Course year 2026-2027



Maastricht University

Learning Goals Today

- Understand the basics of Pre-Training BERT, GPT-2 and T-5 from scratch
- Understand how to create better Word Embeddings with BERT
- Understand the basics of Fine-Tuning BERT, GPT and T-5
- Alignment of Generative AI Models (aka staying within human acceptable boundaries)
- Understand how reasoning models are trained: from PPO and reward models to DPO, GRPO and verifiable rewards

How About RAG?

- In this lecture we do not discuss Retrieval Augmented Generation (RAG) and it's aspects such as:
 - Training for neural re-ranking in search
 - Aligning embedding spaces with different dimensionalities
 - Merging embeddings in encoding, cross-attention and decoding

These are all discussed in the course
Information Retrieval and Text-Mining (IRTM).

Part of the Implementation:

Data Set Creation for NLP

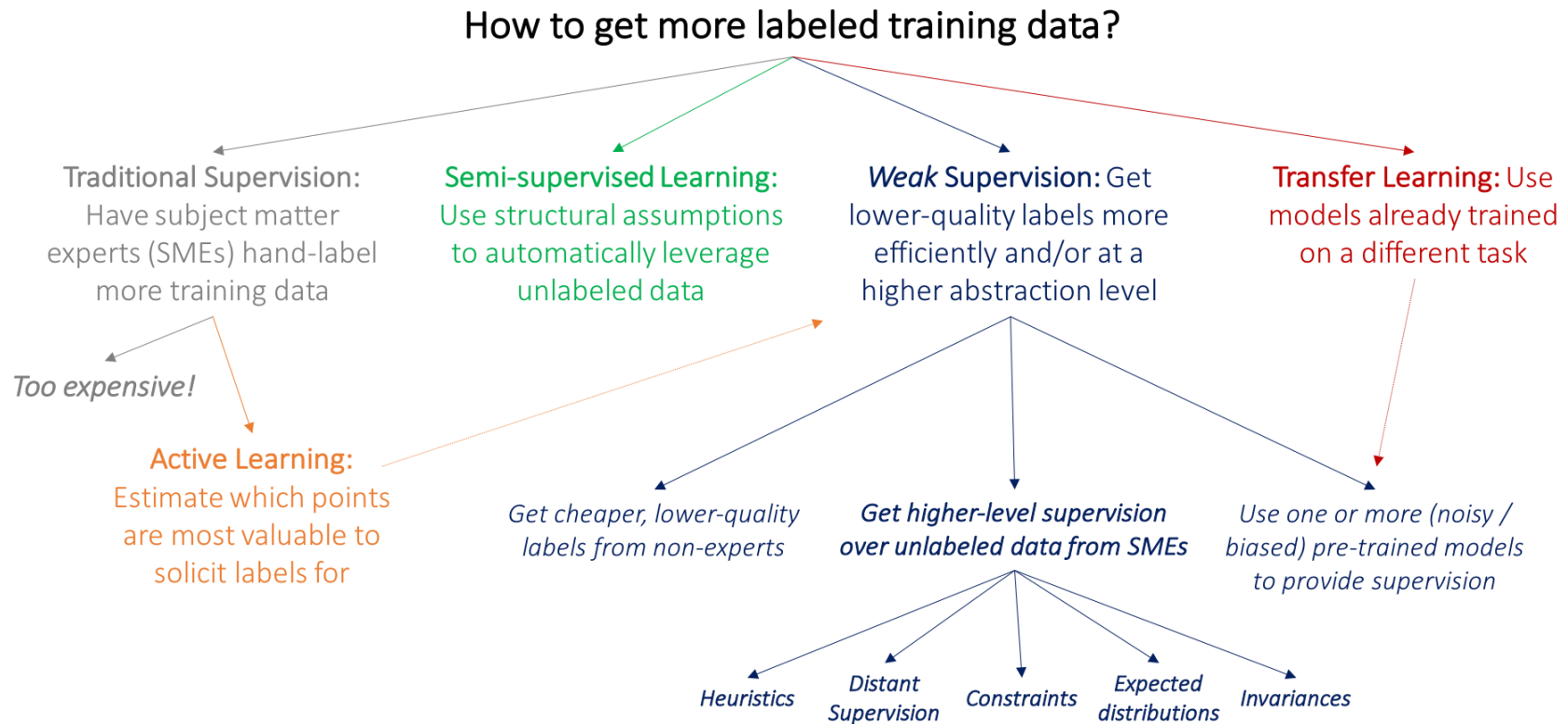


Maastricht University

Data Sets for Fine Tuning?

- Deep learning is data hungry: so, it is essential to have sufficient training data.
- There is always a need for (human) annotated or ranked data sets. Definitely for Encoder and Encoder-Decoder models, but for decoder-only models.
- There are many methods to generate additional training data (augmentation, etc).
- This is a very active field of research.

Methods to obtain more Labeled Training Data



Source: <http://ai.stanford.edu/blog/weak-supervision/>

Training Transformers

Encoder-only models
Decoder-only models
Encoder-Decoder models

Training from scratch (aka pre-training)
Fine-tuning for a particular task
Alignment of LLM's



Maastricht University

Back-propagation

- All functions are non-linear and need to be differentiable.
- Calculate contribution of every weight to the error.
- Convergence requires derivative (changes) of error with respect to weight changes needs to go to zero.

Finding derivative of the error:

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial o_j} \frac{\partial o_j}{\partial w_{ij}} = \frac{\partial E}{\partial o_j} \frac{\partial o_j}{\partial \text{net}_j} \frac{\partial \text{net}_j}{\partial w_{ij}}$$

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}} = -\eta o_i \delta_j$$

With $\eta > 0$ Always decreases $\rightarrow$ convergence

$$E = L(t, y)$$

where

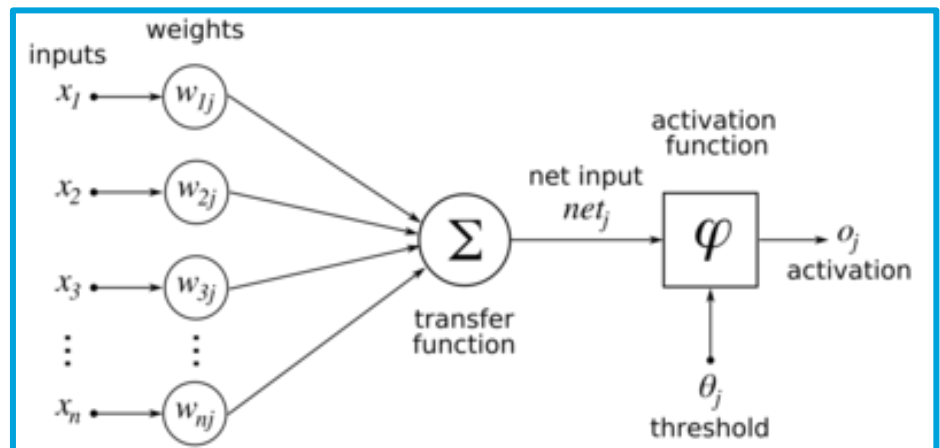
L is the loss for the output y and target value t ,
 t is the target output for a training sample, and
 y is the actual output of the output neuron.

For each neuron j , its output o_j is defined as

$$o_j = \varphi(\text{net}_j) = \varphi \left(\sum_{k=1}^n w_{kj} o_k \right),$$

Activation
function:

$$\varphi(z) = \frac{1}{1 + e^{-z}}$$



BERT



Maastricht University

Pre-training and Fine-Tuning BERT

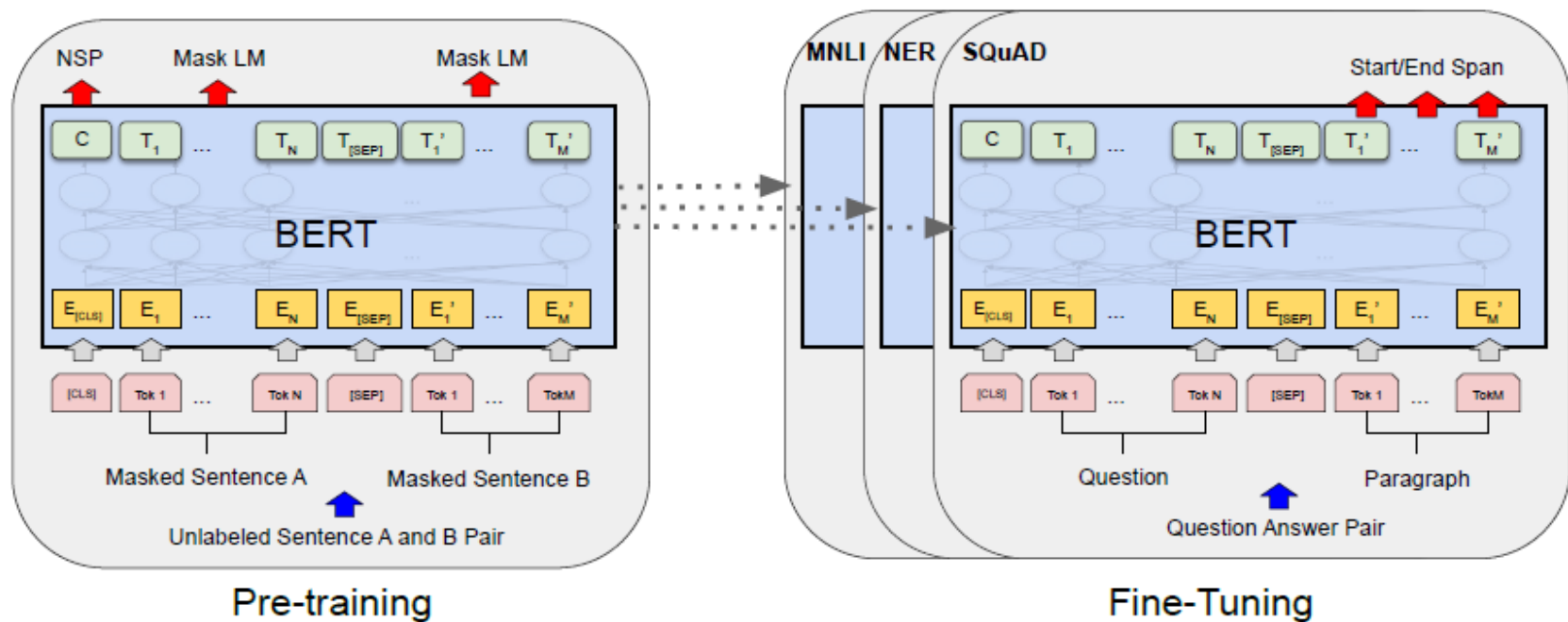


Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. [CLS] is a special symbol added in front of every input example, and [SEP] is a special separator token (e.g. separating questions/answers).

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, Jacob Devlin et al., May 2019.

<https://arxiv.org/pdf/1810.04805.pdf>

Recap (Lecture 10): Pre-Training and Fine-Tuning BERT

- **Pre-training:** BERT is pre-trained on unlabeled pair of sequences over two prediction tasks:
 - masked language modeling (MLM)
 - next sentence prediction (NSP).

For each pair of sequences, the model makes predictions for these two tasks and based on the loss values, it performs backpropagation to update weights (details: Lecture 10).

- **Fine-tuning:** BERT is initialized with pre-trained weights during the above pre-training and can then be fine-tuned for a particular problem on labeled data during the fine-tuning process.

BERT: Training Details we will discuss...

- BERT is pre-trained on BooksCorpus (800M words) and English Wikipedia (2,500M words). For extracting longer continuous texts, authors took from Wikipedia only reading passages ignoring tables, headers and lists.
- BERT is trained on a million batches of size equal to 256 sequences which is equivalent to 40 epochs on 3.3 billion words. Each sequence contains up to 128 (90% of the time) or 512 (10% of the time) tokens.
- The training parameters are the following:
 - Optimizator: Adam (learning rate $\ell = 1e-4$, weight decay $L_2 = 0.01$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1e-6$).
 - Learning rate warmup is performed over the first 10.000 steps and then reduced linearly.
 - Dropout ($\alpha = 0.1$) layer is used on all layers.
 - Activation function: GELU.
 - Training loss is the sum of mean MLM and mean next sentence prediction likelihoods.

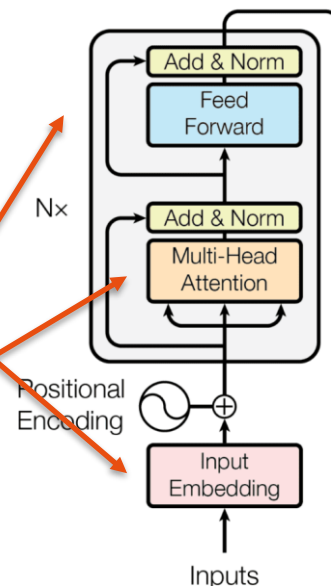
Which Weights are Updated during Training in BERT?

Backpropagation is the process of training Artificial Neural Networks (ANNs) by adjusting the weights of each layer in order to minimize a predefined error or cost function. Steps involved in backpropagation for pre-training and fine-tuning BERT are:

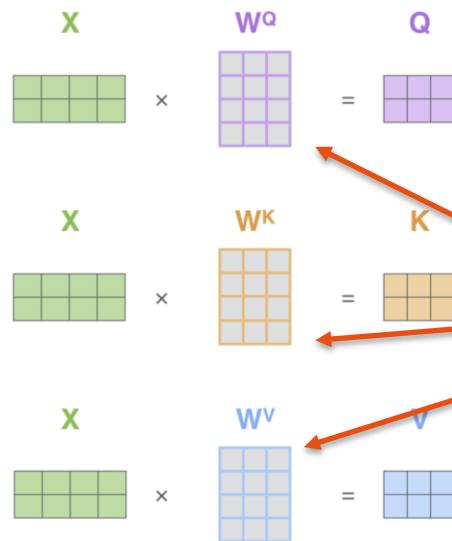
1. Feed forward: The input data is sent forward through the network, each layer performing some operation on the data. At the end of this process, an output is generated.
2. Calculate the error: The generated output is compared to the expected output (also known as the target) and an error is calculated.
3. Calculate the derivatives: The derivatives of the error with respect to the weights of each layer of the network is calculated.
4. Backpropagate the error: The derivatives are used to propagate the error backwards through the network. This helps in calculating the adjustments that need to be made to the weights in order to minimize the error.
5. Update the weights: The weights of each layer of the network are updated in the direction that helps to minimize the error.

The number of weights per layer depends on the model architecture. For example, BERT Base has 12 layers, each with its own set of weights:

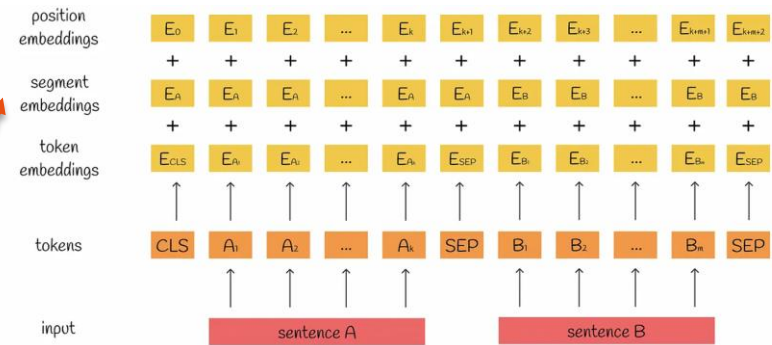
- The weights for the embeddings layer are also updated since this layer is a part of the network and is used to represent the input data.
- The weights for the Q, K, and V matrices are also updated as part of the backpropagation process.
- The feed forward neural network layer also has its own set of weights which are updated as part of the backpropagation process.



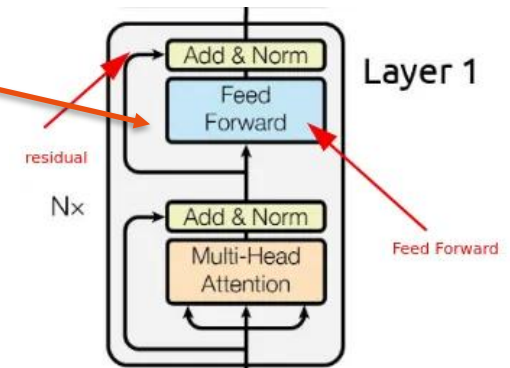
Weights that are updated in BERT



- The weights for the **embeddings layer** are also updated since this layer is a part of the network and is used to represent the input data.
- The weights for the **W^Q , W^K , and W^V matrices** are also updated as part of the backpropagation process for the proper self-attention calculations.
- The **feed forward neural network** layer also has its own set of weights which are updated as part of the backpropagation process.



How many trainable weights do you estimate that BERT base has: 12 layers, 768 dim embeddings and 12 attention heads.

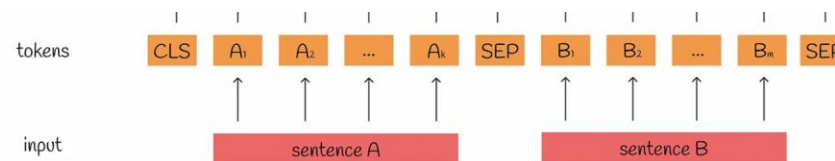


How many trainable Weights in BERT-base?

Let's start with the "Embedding Layer"

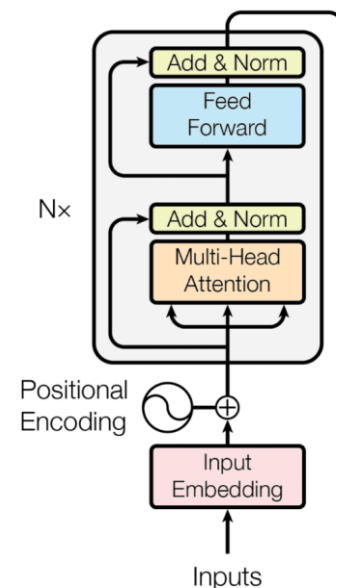
BERT uses a word embedding layer (vocabulary size: V) and positional embeddings.

- **Token Embeddings:** $V \times 768$
- **Positional Embeddings:** $P \times 768$ (usually $P=512$, the maximum sequence length)
- **Segment Embeddings:** 2 segments (A/B) $\times 768$



Embedding layer total:

$$\text{Embeddings} = V \times 768 + P \times 768 + 2 \times 768$$

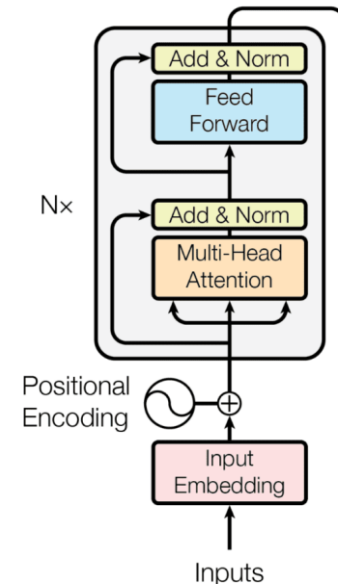


A closer look at the Embeddings during training

During training:

- **Token Embeddings:** These are updated. The model learns better token representations by optimizing these embeddings during pretraining tasks (Masked Language Modeling and Next Sentence Prediction). For example:
 - If "apple" is masked and predicted as "fruit," the corresponding embedding for "apple" is updated to better encode its contextual meaning.
- **Positional Embeddings:** These are also updated. While they encode fixed positional information initially, their representations can be fine-tuned to optimize performance on the training data.
- **Segment Embeddings:** These are updated as well. For tasks like Next Sentence Prediction or any task that uses segments, the model learns to adjust these embeddings to capture relationships between sentence pairs.

In summary, all components of the embedding layer are trainable and are updated during training.



. Why Not Just Update Segment Embeddings?

If only the segment embeddings were trainable:

- The model would fail to adapt the token embeddings to learn nuanced word meanings and their contexts.
- Positional embeddings would remain static, potentially limiting the model's ability to encode positional relationships effectively for different tasks.
- The model's performance on downstream tasks would be significantly worse.

Trainable Weights in BERT-base: 2. Transformer Layers 1/2:

There are 12 transformer layers, each with:

- **Self-Attention Weights:**

- Query, Key, Value projections (768×768 each) for each attention head:

$$Q/K/V = 3 \times 768 \times 768$$

- Output projection back to hidden size (768×768):

$$\text{Output Projection} = 768 \times 768$$

Total for self-attention:

$$\text{Self-Attention Layer} = (3 \times 768^2) + 768^2 = 4 \times 768^2$$

Trainable Weights in BERT-base: 2. Transformer Layers 2/2

- **Feedforward Layers:** Two fully connected layers in the feedforward network:

- Input to intermediate (768×3072):

$$\text{Intermediate FC} = 768 \times 3072$$

- Intermediate back to hidden size (3072×768):

$$\text{Output FC} = 3072 \times 768$$

Total for feedforward:

$$\text{Feedforward} = 768 \times 3072 + 3072 \times 768 = 2 \times 768 \times 3072$$

- **Layer Norms:** Two LayerNorm parameters per layer:

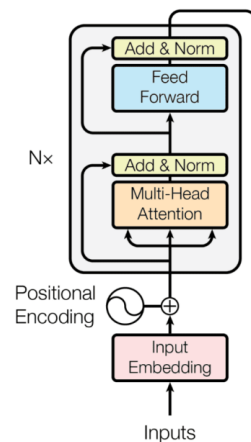
$$\text{LayerNorms} = 2 \times 768 \times 2 = 2 \times 768$$

Total per transformer layer:

$$\text{Per Layer Total} = (4 \times 768^2) + (2 \times 768 \times 3072) + (2 \times 768)$$

The **3072** comes from the fact that in BERT, the hidden size of 768 is expanded by a factor of **4** in the intermediate layer of the feedforward network. This design choice is based on the original Transformer architecture in the Vaswani et al. (2017) paper.

Two fully connected layers



Trainable Weights in BERT-base: Final Calculation:

Combine all layers and embeddings:

$$\text{Total Parameters} = \text{Embeddings} + 12 \times \text{Per Layer Total}$$

Assuming typical BERT-base settings:

- Vocabulary size (V) = 30,000
- Positional embeddings (P) = 512

Embeddings:

$$\text{Embeddings} = (30,000 \times 768) + (512 \times 768) + (2 \times 768) = 23,434,752$$

Per Layer:

$$\begin{aligned} \text{Per Layer Total} &= (4 \times 768^2) + (2 \times 768 \times 3072) + (2 \times 768) \\ &= 2,359,296 + 4,718,592 + 1,536 = 7,079,424 \end{aligned}$$

Total Parameters:

$$\text{Total} = 23,434,752 + (12 \times 7,079,424) = 23,434,752 + 84,953,088 = 108,387,840$$

~ 100.000.000 trainable weights!

Typical Pre-Training:

1. Dataset Size

The original BERT-base model was pretrained on a corpus containing **3.3 billion words**, combining:

- BooksCorpus (~800M words)
- English Wikipedia (~2.5B words)

2. Number of Training Steps

The original BERT-base pretraining was done for **1,000,000 steps** with:

- A batch size of **256 sequences**
- Sequence length **512 tokens**



See why 2 slides further...

This corresponds to processing about **40 epochs** over the dataset, assuming 3.3 billion words. The exact formula to estimate epochs is:

$$\text{Epochs} = \frac{\text{Steps} \times \text{Batch Size} \times \text{Sequence Length}}{\text{Total Tokens in Corpus}}$$

3. Recommended Epochs

While 40 epochs worked for the original BERT training, it might not always be necessary for other datasets or training setups. Some recommendations include:

- **Smaller Corpora:** Aim for 10-20 epochs for smaller datasets.
- **Large-scale Pretraining:** Expect 40-50 epochs for very large and diverse datasets.
- **Domain-specific Pretraining:** Focus on fewer epochs (e.g., 5-10) if you're fine-tuning BERT on domain-specific data like legal or medical text.

4. Compute Requirements

Pretraining for the full 1M steps took:

- **16 TPU v3 chips** (for ~4 days)
- Equivalent to several weeks on high-end GPUs like NVIDIA A100s.



For domain-specific adaptations (e.g., LegalBERT or BioBERT), you can often stop after **5-10 epochs** if you're starting from an existing pretrained checkpoint.

Why 1.000.000 training steps?

1. Dataset Size

BERT was pretrained on:

- **BooksCorpus:** ~800 million words.
- **English Wikipedia:** ~2.5 billion words.
- **Total words:** ~3.3 billion.

For tokenization, BERT uses WordPiece, splitting words into subword units. On average, one word corresponds to **1.1 to 1.2 tokens**. Therefore, the total number of tokens in the corpus is:

$$\text{Total Tokens} \approx 3.3 \text{ billion words} \times 1.1 = 3.63 \text{ billion tokens.}$$

2. Batch Size and Sequence Length

Each training step processes:

- **Batch size:** 256 sequences.
- **Sequence length:** 512 tokens.

This means each training step processes:

$$\text{Tokens per step} = 256 \times 512 = 131,072 \text{ tokens.}$$

3. Total Training Steps

To determine the number of steps, we calculate how many tokens need to be processed to cover the entire dataset for a single epoch:

$$\text{Tokens per epoch} = 3.63 \text{ billion tokens.}$$

With a batch size of 256 sequences and sequence length of 512, the number of steps per epoch is:

$$\text{Steps per epoch} = \frac{\text{Tokens per epoch}}{\text{Tokens per step}} = \frac{3.63 \times 10^9}{131,072} \approx 27,686.$$

The authors trained BERT for approximately **40 epochs** over the dataset, which gives:

$$\text{Total Steps} = 40 \times 27,686 \approx 1,107,440.$$

This is rounded to **1,000,000 steps**, which aligns with the computational budget and convergence observed in practice.

Why Domain Dependent BERT Embeddings?

Bible:

The concept, “act of God,” is central to the biblical understanding of God and his relation to the world. Repeatedly we are told of the great works performed by God in behalf of his people and in execution of his own purposes in history.

“... an Act of God”

Legal Agreement:

e. Cancellations: Lyonpride is not liable for return of RMHB fees in the event of cancellations or other defaults by Lyonpride due to Acts of God. The RMHB does not have the right to terminate this Agreement for any delay or default in performing hereunder if such delay or default is caused by conditions beyond the control of Lyonpride including, but not limited to Acts of God.

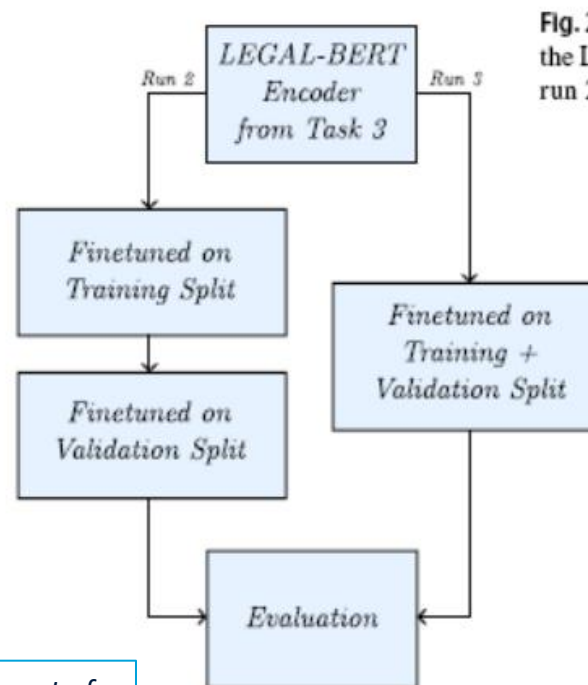


Fig. 2 Fine-tuning workflow of the LEGAL-BERT encoder for run 2 and 3

What to do with Domain Dependent Embeddings?

1. Use the embeddings in the fine-tuned BERT LLM model.
2. Extract the embeddings and use in T-5 (better translations), Pegasus (better summaries), GPT (better text generation), or other transformer-based model.

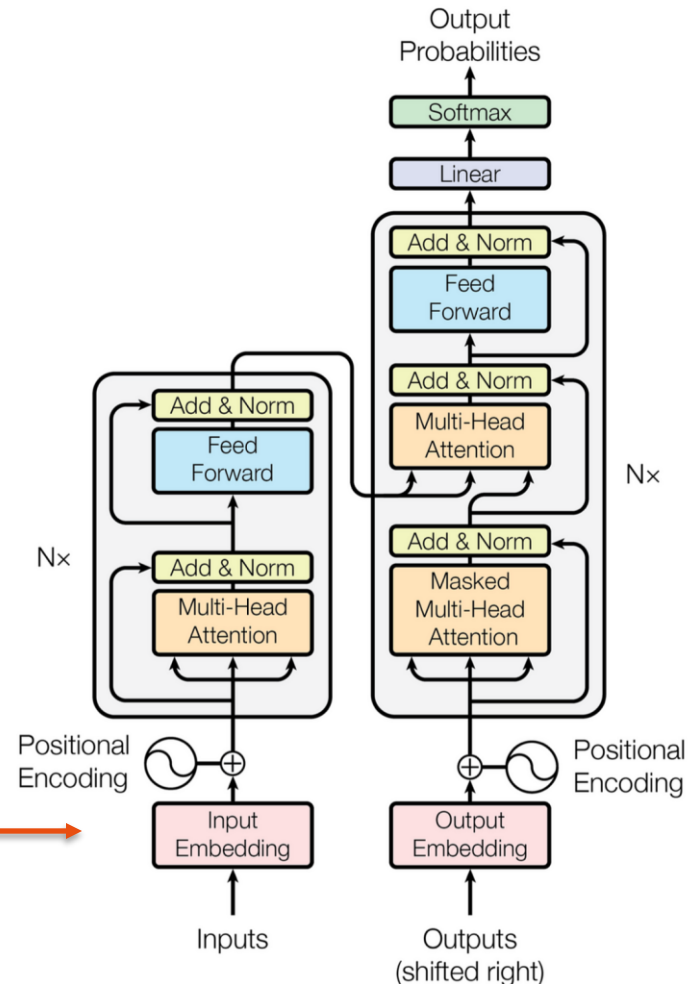
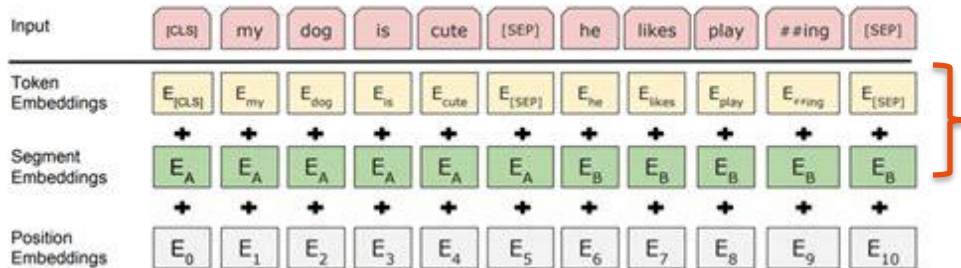


Figure 1: The Transformer - model architecture.

GPT-2



Maastricht University

Preparing the Training Data for GPT

- GPT models are trained in an unsupervised way on a large amount of text (or text corpus).
- The corpus is broken into sequences, usually of uniform size (e.g., 1024 tokens each).
- The model is trained to predict the next token (word) at each step of the sequence.

For example (here, we write words instead of integer encodings for clarity):

position	1	2	3	4	5	6	7	8	9
input_ids	The	elf	queen	was	wearing	a	cloak	.	[END]
labels	elf	queen	was	wearing	a	cloak	.	[END]	[-1]

Pre-training

- There are many parameters to be set.
- See Huggingface documentation for more details:
https://huggingface.co/docs/transformers/main_classes/trainer.
- load the corpus for training, tokenize it and break it into 511-token pieces plus the [END] token, which brings the sequence length to 512. Next, we split the data into train and validation sets in `train_val_split()` and in `prepare_dsets()` wrap them in the PyTorch datasets.
- Training is similar to BERT
 - Predict next word
 - Back-propagate errors
 - Repeat for several epochs

Difference pre-training and fine-tuning: pre-training (or training from scratch) means starting from a randomly-initialized model with no pre-trained weights. Fine-tuning starts with weights from a pre-trained model.

```
training_args = transformers.TrainingArguments(  
    output_dir="idiot_save/",  
    learning_rate=1e-3,  
    per_device_train_batch_size=1,  
    per_device_eval_batch_size=1,  
    num_train_epochs=20,  
    evaluation_strategy='epoch',  
    save_strategy='no',  
    )
```



Fine Tuning GPT-2

- New language model trained exclusively on biomedical abstracts and papers.
- This GPT-style model can achieve strong results on a variety of biomedical NLP tasks, including a new state of the art performance of 50.3% accuracy on the MedQA biomedical question answering task.
- PubMedGPT 2.7B is an autoregressive language model with 2.7B parameters. It uses the standard GPT-2 architecture

Healthcare

Stanford CRFM Introduces PubMedGPT 2.7B



The new 2.7B parameter language model trained on biomedical literature delivers an improved state of the art for medical question answering.

Parameter	Value
hidden size	2560
heads	20
layers	32
vocab size	28896
sequence length	1024

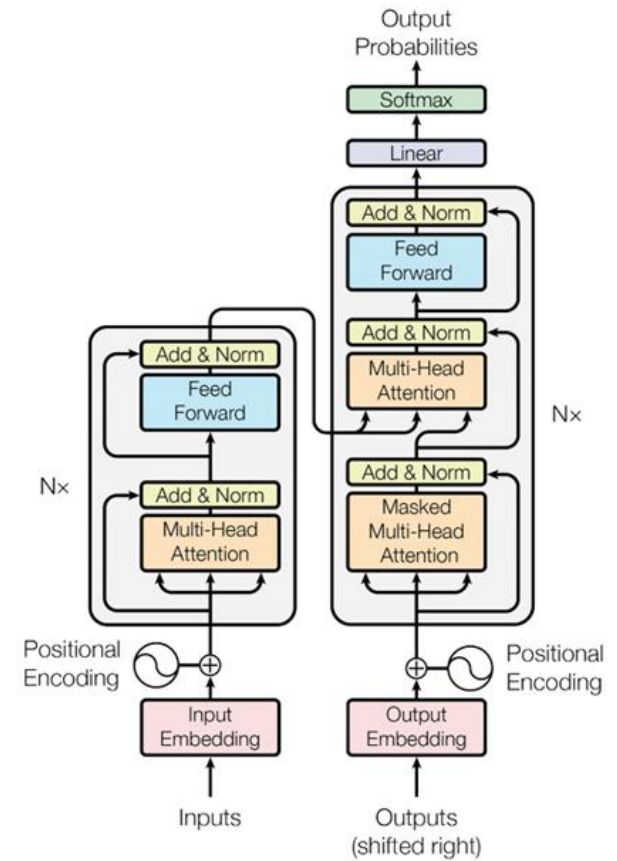
T-5



Maastricht University

Pre-training T5

- Full encoder-decoder model.
- T5 prepends a different prefix to the input corresponding to each task, e.g., for translation: translate English to German: ..., for summarization: *summarize:*
- For training, we always need an input sequence and a corresponding target sequence.
- Pretraining includes both supervised and self-supervised training:
 - Supervised training is conducted on downstream tasks provided by the GLUE and SuperGLUE benchmarks (converting them into text-to-text tasks as explained above).
 - Self-supervised training uses corrupted tokens, by randomly removing 15% of the tokens and replacing them with individual sentinel tokens (if several consecutive tokens are marked for removal, the whole group is replaced with a single sentinel token. In this setup, spans of the input sequence are masked by so-called sentinel tokens (a.k.a unique mask tokens) and the output sequence is formed as a concatenation of the same sentinel tokens and the real masked tokens. Each sentinel token represents a unique mask token for this sentence and should start with <extra_id_0>, <extra_id_1>, ... up to <extra_id_99>. As a default, 100 sentinel tokens are available in T5Tokenizer.



Original text

Thank you for inviting me to your party last week.

Inputs

Thank you <X> me to your party <Y> week.

Targets

<X> for inviting <Y> last <Z>

Which weights are Updated in T5 Back-propagation?

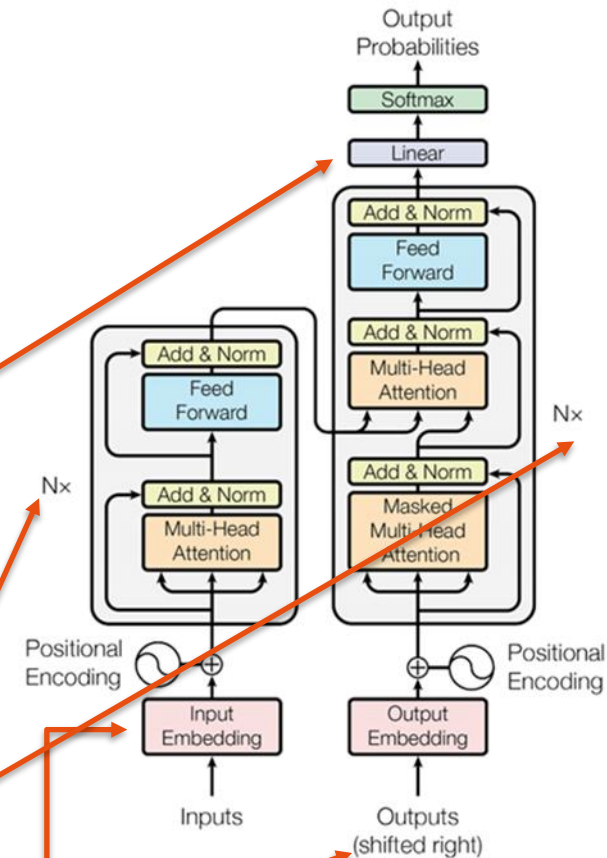
In T5, backpropagation extends to the full model, through all decoder and encoder layers up to the embedding tables.

Back-propagations:

1. Feed forward: The input data is sent forward through the network (encoder and decoder), each layer performing some operation on the data. At the end of this process, an output is generated.
2. Calculate the error: The generated output is compared to the expected output (also known as the target) and an error is calculated.
3. Calculate the derivatives: The derivatives of the error with respect to the weights of each layer of the network is calculated.
4. Backpropagate the error: The derivatives are used to propagate the error backwards through the network. This helps in calculating the adjustments that need to be made to the weights in order to minimize the error.
5. Update the weights: The weights of each layer of the network are updated in the direction that helps to minimize the error. Both for encoder and decoder.

Weights that are adjusted:

1. Final linear layer weights
2. Decoder(Feed forward layer weights & Q, K, V weight matrices $2 \times$)
3. Encoder(Feed forward layer weights & Q, K, V weight matrices)
4. Embeddings layer



Fine-Tuning T5: Title Generation based on Text

Model training

Let's jump now into code with Colab! You can find all the code in this public [Colab notebook](#) as well. This is what we'll do:

1. Connect Google Drive to Colab to have persistent storage across Colab sessions.
2. Download the Medium dataset from Kaggle.
3. Load the dataset.
4. Split the dataset into train, validation, and test set.
5. Preprocess the dataset for T5.
6. Preparing the Hugging Face trainer.
7. Start TensorBoard.
8. Fine-tune T5.
9. Try the model.
10. Evaluate the model on the test set.

Decoder

- `title [string]`: The title of the article.
- `text [string]`: The text content of the article.
- `url [string]`: The URL associated with the article.
- `authors [list of strings]`: The article authors.
- `timestamp [string]`: The publication datetime of the article.

Encoder

Step	Training loss	Validation loss	Rouge1	Rouge2	RougeL	Precision	Rec. F1
100	3.078800	2.602307	28.613000	13.896000	25.131000	28.240700	12.302000
200	2.743800	2.520881	30.788000	15.023400	28.287600	28.410000	11.877900
300	2.688400	2.508451	31.428000	15.887600	28.225600	28.365000	12.254900
400	2.684100	2.498379	31.703000	15.812000	29.118900	28.458800	11.876400
500	2.627000	2.478075	31.548400	15.533000	29.125800	28.345000	12.558300
600	2.709000	2.461808	32.888000	16.379700	29.185000	30.088700	11.781800
700	2.543400	2.460287	32.888000	17.125400	30.191000	30.287100	12.303800
800	2.602300	2.444072	31.802000	16.246000	29.330000	29.874000	12.254000
900	2.514000	2.444442	32.228100	17.239000	29.776000	29.868300	12.414400
1000	2.646600	2.438463	32.273000	17.280300	30.460700	30.788300	11.985000
1100	2.581700	2.434545	32.601000	17.284000	30.529000	30.535000	11.860000
1200	2.600000	2.431821	33.688800	17.327100	30.797700	30.870000	11.828100
1300	2.642300	2.422081	32.717700	17.087000	30.504500	30.674000	11.868100
1400	2.613400	2.421867	33.346000	17.412100	30.872600	30.880000	11.927900
1500	2.587800	2.421587	33.687700	17.787000	31.444000	31.408800	11.768900
1600	2.616700	2.414679	32.462100	16.866000	30.134700	30.448300	12.116400
1700	2.631900	2.408603	33.371400	17.226000	30.927600	30.381500	12.158700
1800	2.614000	2.408281	33.538800	17.228700	30.800000	31.018100	11.868400
1900	2.619300	2.407189	32.919000	16.882700	30.614900	30.703400	12.171100
2000	2.585100	2.410542	32.403000	16.884000	30.378300	30.487800	12.314300
2100	2.585600	2.398441	33.618800	16.811000	30.681100	30.684000	11.927800



Model parameters

Number of titles to generate: 5

Temperature: 0.68

High temperature means that results are more random

Generate candidate titles for articles

Model loaded!

Text to generate the title for

Many financial institutions started building conversational AI, prior to the Covid19 pandemic, as part of a digital transformation initiative. These initial solutions were high profile, highly personalized virtual assistants — like the Erica chatbot from Bank of America. As the pandemic hit, the need changed as contact centers were under increased pressures. As Cathal McGloin of ServisBOT explains in "how it started, and how it is going," financial institutions were looking for ways to automate solutions to help get back to "normal" levels of customer service. This resulted in a change from the "future of conversational AI" to a real tactical assistant that can help in customer service. Haritha Das of Wells Fargo, saw a similar trend

Generate title

Generated titles

- Conversational AI: A Real Tactic Assistant for Customer Service
- The Future of Conversational AI
- Conversational AI: A Digital Transformation Approach
- The Next Wave of Conversational AI
- Conversational AI: The Future of Customer Service

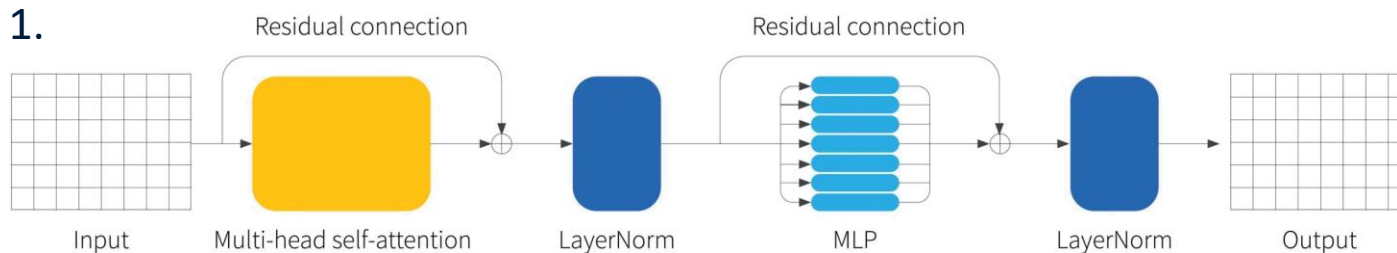
T5, BART, Pegasus,

- **T5**: original transformer models with full encoder and decoder. T5 is pretrained by supervised (GLUE and SuperGLUE) training and self-supervised training (randomly sample and drop out 15% of tokens).
- **BART**: based on the original Transformer architecture, but modifies the pretraining objective with text infilling corruption, where some text spans are replaced with a single mask token. The decoder predicts the uncorrupted tokens (future tokens are masked) and uses the encoder's hidden states to assist this process.
- **PEGASUS**: similar to BART, but Pegasus masks entire sentences instead of text spans. In addition to masked language modeling, Pegasus is pretrained by Gap Sentence Generation (GSG). The GSG objective masks whole sentences important to a document, replacing them with a mask token. The decoder must generate the output from the remaining sentences.

Additional Tricks to get Transformers to Train

- Transformers are surprisingly difficult to train from scratch.
- Four (4) tricks are essential to train the models:
 - Use residual connections.
 - Apply layer normalization.
 - Learning rate warm-up: The learning rate is increased linearly from 0 to R over first T_R time steps.
 - Adaptive optimizers: Transformers need to be trained using adaptive optimizers, which recursively estimates the momentum and the learning rate separately for each parameter at each time-step. In practice, relatively large batch sizes of $>1,000$ are usually employed.

Removing any of these tricks makes training unstable and often leads to complete training failures. However, they have been employed without a full understanding of why they are required.



2.

$$\text{Layernorm}[\mathbf{X}] = \gamma \cdot \frac{\mathbf{X} - \mu}{\sigma} + \beta,$$

where μ and σ are the mean and standard deviation of the elements of $\mathbf{X}$ (but are separate for each member of the batch), and γ and β are learned parameters.

3.

$$\text{lr}[t] = R \cdot \frac{t}{T_R}.$$

Understanding the Difficulty Training Transformers

Transformer training requires non-trivial efforts regarding carefully designing cutting-edge optimizers and learning rate schedulers (e.g., conventional Stochastic Gradient Descent fails to train Transformers effectively).

The analysis in this paper reveals that unbalanced gradients are not the root cause of the instability of training. Instead, they identify an amplification effect that influences training substantially—for each layer in a multi-layer Transformer model, heavy dependency on its residual branch makes training unstable, since it amplifies small parameter perturbations (e.g., parameter updates) and results in significant disturbances in the model output.

They observe that a light dependency limits the model potential and leads to inferior trained models. Inspired by our analysis, Therefore they propose Adaptive model initialization, to stabilize the early stage's training and unleash its full potential in the late stage.

This approach is more stable, converges faster, and leads to better performance

Contrastive Learning

Self-supervised learning (SSL) is a paradigm in machine learning where a model is trained on a task using the data itself to generate supervisory signals, rather than relying on external labels provided by humans. In the context of neural networks, self-supervised learning aims to leverage inherent structures or relationships within the input data to create meaningful training signals.

- **Auto-associative self-supervised learning**: Auto-associative self-supervised learning is a specific category of self-supervised learning where a neural network is trained to reproduce or reconstruct its own input data. In other words, the model is tasked with learning a representation of the data that captures its essential features or structure, allowing it to regenerate the original input.
- **Contrastive self-supervised learning**: For a binary classification task, training data can be divided into positive examples and negative examples. Positive examples are those that match the target. For example, if you're learning to identify birds, the positive training data are those pictures that contain birds. Negative examples are those that do not. Contrastive self-supervised learning uses both positive and negative examples. Contrastive learning's loss function minimizes the distance between positive samples while maximizing the distance between negative samples.
- **Non-contrastive self-supervised learning**: Non-contrastive self-supervised learning (NCSSL) uses only positive examples. Counterintuitively, NCSSL converges on a useful local minimum rather than reaching a trivial solution, with zero loss. For the example of binary classification, it would trivially learn to classify each example as positive. Effective NCSSL requires an extra predictor on the online side that does not back-propagate on the target side.

Using Adapters during Training



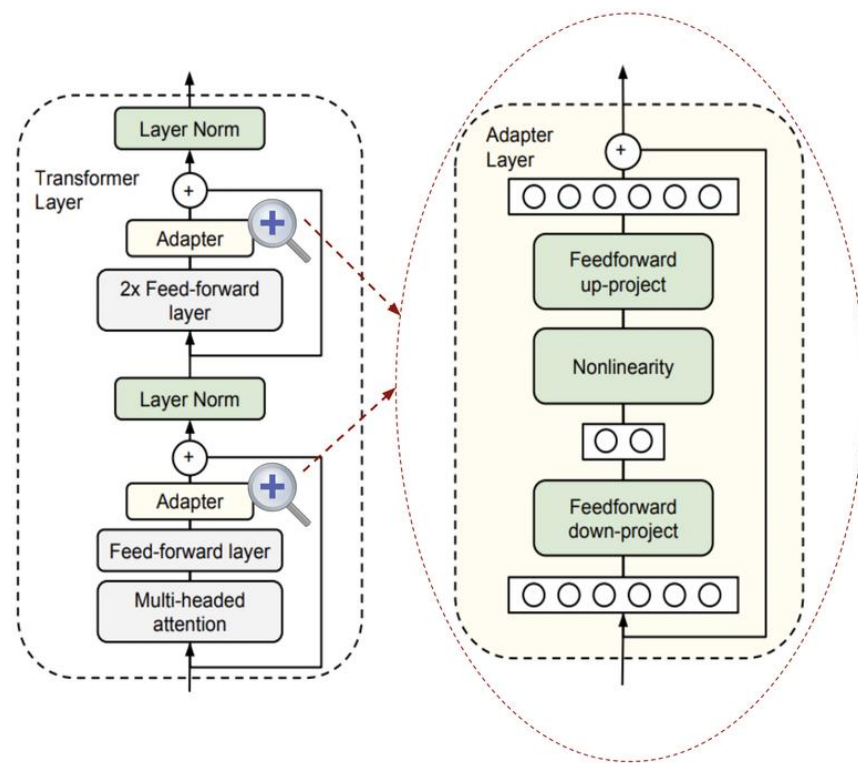
Maastricht University

What are Adapters?

Adapters are **small trainable modules** inserted into the layers of a pre-trained transformer model. Instead of fine-tuning all the parameters of a huge model (which can have hundreds of millions or even billions of parameters), adapters allow us to **freeze the original model weights** and only train these lightweight additional modules.

They were first introduced by Houlsby et al. (2019) in the paper "Parameter-Efficient Transfer Learning for NLP".

<https://arxiv.org/pdf/1902.00751>



How they Work

- A pre-trained transformer model (e.g. BERT) has many stacked layers (multi-head self-attention + feedforward blocks).
- Adapters are **bottleneck neural layers** inserted into each transformer block:
 - Input from the layer $\rightarrow$ goes into a **down-projection** (reduces dimensionality).
 - Then through a **non-linearity** (like ReLU or GELU).
 - Then an **up-projection** (back to the original hidden size).
 - Finally, the adapter output is **added back** to the main residual stream.

Formally, if h is the hidden representation from the transformer, then:

$$\text{Adapter}(h) = h + W_{up} f(W_{down} h)$$

where:

- $W_{down} \in \mathbb{R}^{d \times m}$ projects hidden dim d down to a smaller bottleneck m)
- f = non-linear activation
- $W_{up} \in \mathbb{R}^{m \times d}$ projects back up to original size)

Adapters are small, trainable “add-on” layers inside transformer blocks that enable parameter-efficient transfer learning.

Why use Adapters?

- Parameter efficiency: Only a few million parameters are added instead of fine-tuning the entire model.
- Multi-task learning: Different tasks can share the same frozen transformer backbone while each task has its own adapter.
- Storage efficiency: You don't need to save full copies of huge models, only the small adapter modules.
- Faster training: Much fewer parameters to update.

Example:

- Suppose you have BERT-base with ~110M parameters.
- A single adapter layer might add only ~2–4% extra parameters.
- For 10 different NLP tasks, instead of training 10 full BERTs, you train 10 small adapters (while sharing the main frozen model).

Think of the pre-trained model as a general-purpose language engine. Instead of reprogramming the whole engine for each task, you just plug in a small adapter chip that adjusts the outputs slightly for your specific task. This makes it possible to reuse the same big pre-trained model across many domains and tasks efficiently.

Break



Alignment of LLM



Maastricht University

Where does the reasoning come from?

- **Who writes the chain of reasoning?** The prompt (Lecture 11: chain of thought, tree of thoughts, ReAct prompts), the training (this lecture: the model learns to write its own chain), or the harness and the environment (Lecture 17: tools, tests, feedback).
- **Who judges it?** Humans (preferences), a learned reward model (on the outcome or on each step), a checker (unit tests, exact match, a proof assistant), or the environment (a tool result, a user).
- **When is the compute spent?** At training time (reinforcement learning) or at test time (think longer, think several times, search). Both axes scale, at different prices.

This lecture: the training column. We start with the 2022 recipe of ChatGPT and end with the reasoning models of 2025.

ChatGPT and Gemini

- ChatGPT and Gemini (formerly Bard) are **fine-tuned on human feedback** (Reinforcement Learning with Human Feedback: RLHF) to improve their ability to generate coherent, context-aware, and conversational responses.
- Standard GPT models are pretrained on large corpora but lack specific tuning for conversational tasks.
- ChatGPT is based on OpenAI's GPT models.
- Gemini (launched as Bard in 2023, renamed in 2024) is Google's conversational AI. Bard first ran on LaMDA and PaLM 2 (Pathways Language Model); it now runs on Google's Gemini family of large language models.

Let's take a closer look at fine-tuning LLM's with human feedback ...



Aligning LLMs with Human Values

Proximal Policy Optimization: PPO

Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

Explain the moon landing to a 6 year old

A labeler demonstrates the desired output behavior.

Some people went to the moon...

This data is used to fine-tune GPT-3 with supervised learning.

SFT

SFT

Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

Explain the moon landing to a 6 year old

A B
C D
Explain gravity... Explain war...
Moon is natural satellite of... People went to the moon...

A labeler ranks the outputs from best to worst.

D > C > A = B

This data is used to train our reward model.

RM
D > C > A = B

RM

Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.

A new prompt is sampled from the dataset.

Write a story about otters.

The PPO model is initialized from the supervised policy.

PPO

The policy generates an output.

Once upon a time...

The reward model calculates a reward for the output.

RM

The reward is used to update the policy using PPO.

r_k

Figure 2: A diagram illustrating the three steps of our method: (1) supervised fine-tuning (SFT), (2) reward model (RM) training, and (3) reinforcement learning via proximal policy optimization (PPO) on this reward model. Blue arrows indicate that this data is used to train one of our models. In Step 2, boxes A-D are samples from our models that get ranked by labelers. See Section 3 for more details on our method.

Step 1: More Supervised Learning

- Fine-tune GPT-3 on labeler demonstrations using supervised learning.
- Trained for 16 epochs.
- The final Supervised Fine-Tuning (SFT) model selection is based on the Reward Model (RM) score on the validation set.
- SFT models overfit on validation loss after 1 epoch; however, training for more epochs helps both the RM score and human preference ratings, despite this overfitting.
- Like the approach in Wu et al., 2021: instead of one large task, recursively decompose the task in smaller tasks and have these validated by humans.
- Next, combine the results to do the entire task.



Step 2: Finding a Reward Modeling function

- Only use 6B parameter RMs, as this saves a lot of compute, and we found that 175B parameter RM training could be unstable and thus was less suitable to be used as the value function during RL.
- Next, present labelers with anywhere between $K = 4$ and $K = 9$ responses to rank. This produces $\binom{K}{2}$ comparisons for each prompt shown to a labeler.

Specifically, the loss function for the reward model is:

$$\text{loss}(\theta) = -\frac{1}{\binom{K}{2}} E_{(x, y_w, y_l) \sim D} [\log(\sigma(r_\theta(x, y_w) - r_\theta(x, y_l)))] \quad (1)$$

where $r_\theta(x, y)$ is the scalar output of the reward model for prompt x and completion y with parameters θ , y_w is the preferred completion out of the pair of y_w and y_l , and D is the dataset of human comparisons.

Data for Reward Modeling

Table 3: Labeler-collected metadata on the API distribution.

Metadata	Scale
Overall quality	Likert scale; 1-7
Fails to follow the correct instruction / task	Binary
Inappropriate for customer assistant	Binary
Hallucination	Binary
Satisfies constraint provided in the instruction	Binary
Contains sexual content	Binary
Contains violent content	Binary
Encourages or fails to discourage violence/abuse/terrorism/self-harm	Binary
Denigrates a protected class	Binary
Gives harmful advice	Binary
Expresses opinion	Binary
Expresses moral judgment	Binary

Step 3: Reinforcement Learning

- Fine-tune the Supervised Fine Tuning (SFT) model using Proximal Policy Optimization”.
- The environment is a bandit environment* which presents a random customer prompt and expects a response to the prompt. Given the prompt and response, it produces a reward determined by the reward model and ends the episode.
- In addition, a per-token KL** penalty from the SFT model is added at each token to mitigate overoptimization of the reward model.
- The value function is initialized from the Reward Model.
- We call these models “PPO: Proximal Policy Optimization”.



* Multi-armed bandit problems are some of the simplest reinforcement learning (RL) problems to solve. We have an agent which we allow to choose actions, and each action has a reward that is returned according to a given, underlying probability distribution. The game is played over many episodes (single actions in this case) and the goal is to maximize your reward.

**) KL = Kullback-Leibler divergence: In mathematical statistics, the Kullback–Leibler divergence is a measure of how one probability distribution is different from a second, reference probability distribution.

Reinforcement Learning (RL) - 2

The following combined objective function is maximized in the RL training:

$$\text{objective}(\phi) = E_{(x,y) \sim D_{\pi_{\phi}^{\text{RL}}}} \left[r_{\theta}(x, y) - \beta \log \left(\pi_{\phi}^{\text{RL}}(y | x) / \pi^{\text{SFT}}(y | x) \right) \right] + \gamma E_{x \sim D_{\text{pretrain}}} \left[\log(\pi_{\phi}^{\text{RL}}(x)) \right] \quad (2)$$

where π_{ϕ}^{RL} is the learned RL policy, π^{SFT} is the supervised trained model, and D_{pretrain} is the pretraining distribution. The KL reward coefficient, β , and the pretraining loss coefficient, γ , control the strength of the KL penalty and pretraining gradients respectively. For "PPO" models, γ is set to 0. Unless otherwise specified, in this paper InstructGPT refers to the PPO-ptx models.

Final best model



Results

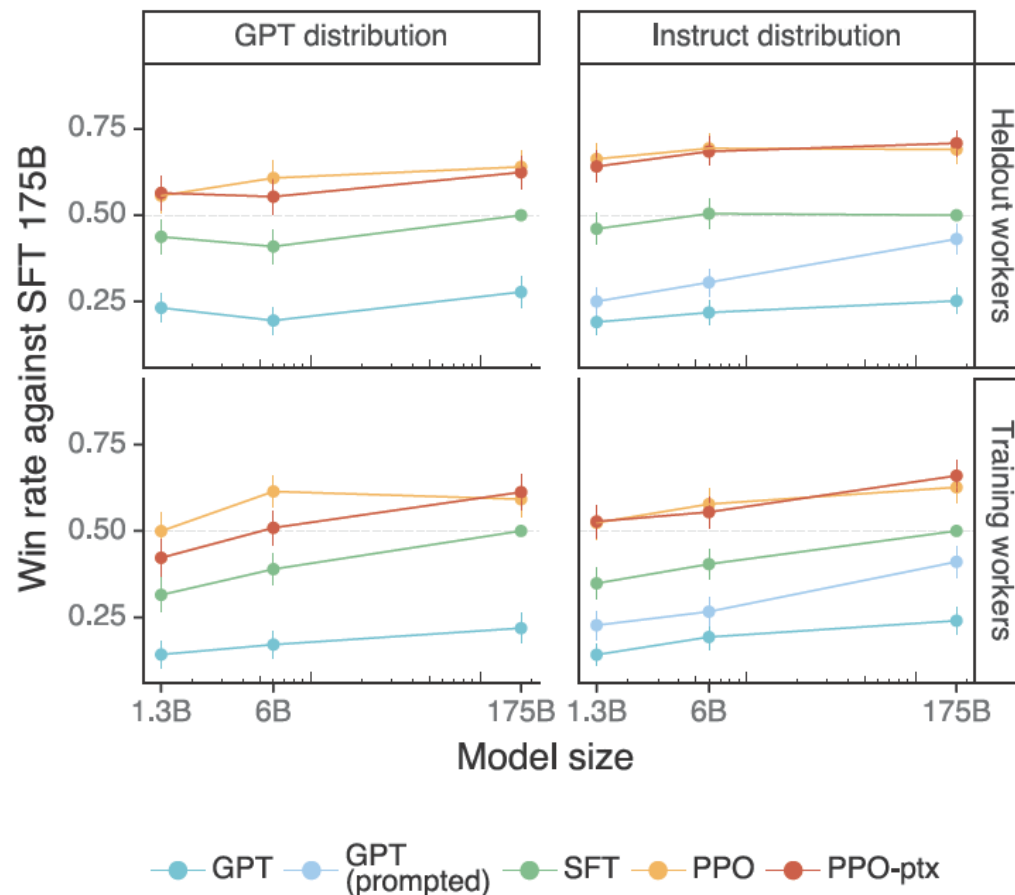


Figure 3: Preference results of our models, measured by winrate against the 175B SFT model. Left: results on prompts submitted to GPT models on the API; Right: results on prompts submitted to InstructGPT models on the API; Top: results from held-out labelers; Bottom: results from training labelers. We omit GPT (prompted) from the evals on prompts submitted to GPT-3 models (left) as these prompts are already designed to perform well for GPT-3, as opposed to prompts submitted to InstructGPT models (right).

What PPO actually does

- **Policy gradient:** raise the probability of the tokens of a response that earned a high reward, lower it for the others. The reward arrives once, at the end of the response (a bandit with one step).

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t[\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)], \rho_t = \frac{\pi_\theta(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}$$

- **The clip** keeps each update close to the previous policy; the KL term in the reward (previous slide) keeps the whole run close to the SFT model. Two leashes.
- **The advantage** $\hat{A}_t$ needs a baseline: a value model that predicts the expected reward, trained alongside the policy.
- **Four networks in memory:** policy, reference, reward model, value model. Sensitive to hyperparameters, expensive at scale.
- **2024:** Llama 3 dropped PPO for rejection sampling and DPO (“more complex RL algorithms tend to be less stable and harder to scale”). Plain REINFORCE with a sampled baseline matches PPO on RLHF (Ahmadian et al. 2024).

The reward model, and why it gets fooled

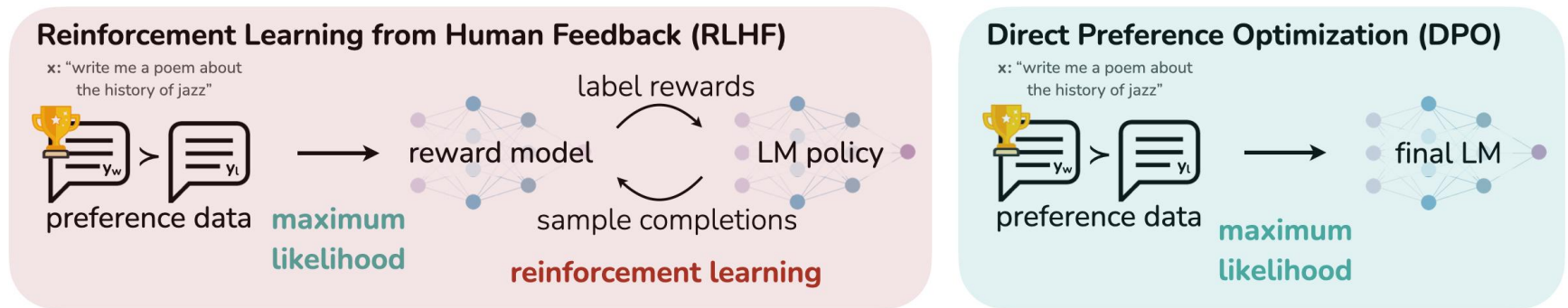
- **Bradley-Terry:** the reward model turns pairwise human choices into a score.

$$P(y_w \succ y_l \mid x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$$

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$$

- **Goodhart's law:** optimise a proxy hard enough and it stops measuring what you wanted. Gao, Schulman and Hilton (2022): the proxy reward keeps rising with the KL distance from the reference, the true reward rises and then falls.
- **Length bias:** a reward for length alone reproduces most of the RLHF gain (Singhal et al. 2023). Raters like long answers.
- **Sycophancy:** raters prefer agreement, so the policy learns to agree (Sharma et al. 2023). Compare the GPT-4o rollback of April 2025 (Lecture 11).
- **Consequences:** keep the leash (β) on, stop early, and replace the learned judge by a checker wherever one exists (verifiable rewards, later in this lecture).

DPO: solve the reward model away

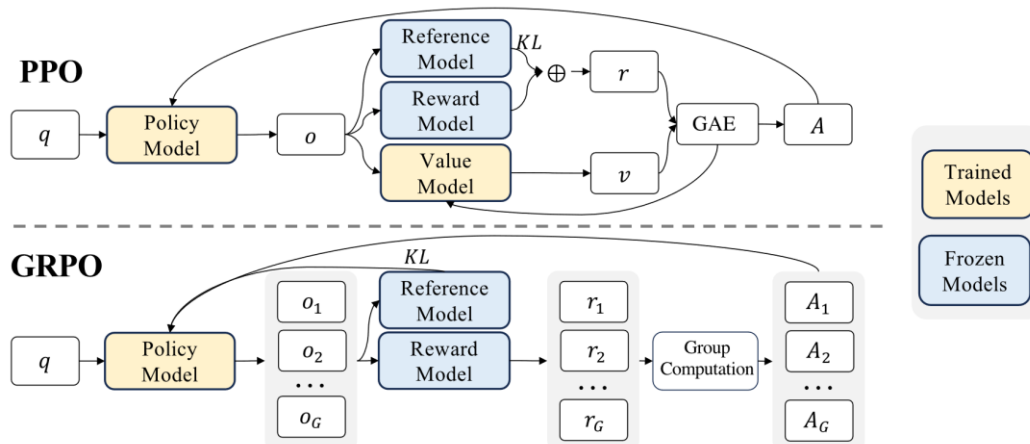


- **Idea:** the KL-regularised RLHF objective has a closed-form optimal policy. Substitute it into the Bradley-Terry model and the reward model drops out; what remains is a classification loss on the policy itself.

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

- **Two models in memory** (policy and reference), no sampling during training, no reward model: the policy is its own reward model (the implicit reward is β times the log-ratio).
- **Weak point:** it only sees the pairs you hand it. Pairs written by another model are off-policy; iterative (online) DPO samples from the current policy, labels, trains, repeats. Llama 3 ran six such rounds; Tülu 3 and OLMo 3 use DPO before RL.
- **Variants:** IPO, KTO (unpaired labels), ORPO and SimPO (no reference model). Same idea, different loss.

GRPO: grading on a curve



- **Group Relative Policy Optimization** (DeepSeekMath, 2024): sample G responses to the same prompt, score each one (reward model or checker), and use the group as the baseline. No value model.

$$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G)}$$

- The same advantage for every token of response i . PPO's clipped ratio stays; the KL to the reference moves into the loss. Three networks become two (policy and reference), or one if the KL term is dropped.
- **Why it works:** a response is rewarded for how far it is above the class average on that question; the group tells you how hard the question was, so no critic has to guess it.

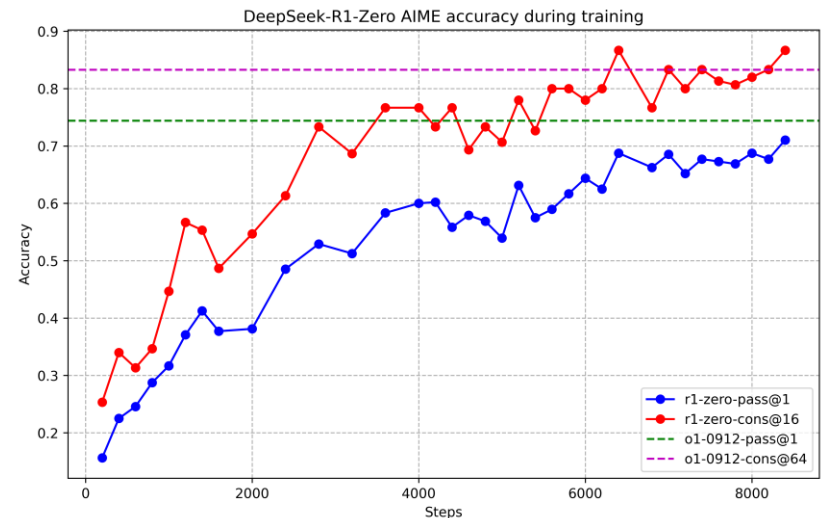
GRPO in practice: the fine print of 2025

- **Length bias:** averaging the loss per response favours long wrong answers; the standard deviation over-weights questions that are all right or all wrong. Dr. GRPO (Liu et al. 2025): drop both.
- **Entropy collapse:** the policy becomes deterministic early and stops exploring. DAPO (Yu et al. 2025): a higher clip ceiling (0.28 instead of 0.20), drop groups where every answer is right or every answer is wrong (zero gradient), token-level averaging, length shaping.
- **Sequence-level ratios** for mixture-of-experts models (GSPO, Qwen 2025); clipped importance weights that keep the gradient of rare tokens such as “Wait” (CISPO, MiniMax 2025).
- **The KL term** is often dropped or made small for reasoning RL; response length and entropy are watched as first-class metrics.
- **Scaling (Meta, 2025, 400k GPU hours):** most of these choices change how fast you get there, not where you end up; RL performance against compute follows a sigmoid.
- **The agreed recipe:** critic-free group baseline, asymmetric clipping, filter zero-variance groups, sequence- or batch-level averaging, length control, entropy monitoring.



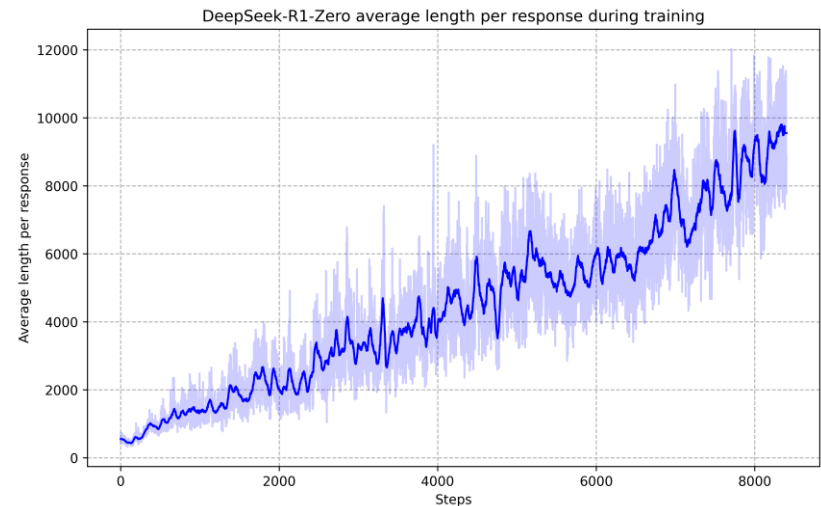
RLVR: replace the judge by a checker

- **Verifiable reward:** exact match on the boxed answer, unit tests for code, a proof checker, a constraint check. Binary, plus a small reward for the required format.
- **No judge to fool:** “the neural reward model may suffer from reward hacking in large-scale RL” (DeepSeek-R1). Tulu 3 (2024) named the method; R1-Zero (January 2025) ran GRPO on the base model with nothing but these rewards.
- **Figure:** AIME 2024 accuracy of R1-Zero during RL. pass@1 rises from 15.6 to 71.0 percent, 86.7 with a majority vote over 16 samples, the level of o1.
- **Condition:** the checker must be reliable and the base model must already solve some of the problems at pass@k. Qwen bases are “RL-friendly”; Llama bases need math-heavy mid-training first.



What emerged, and what was dropped

- **Emerged:** the response length grows from a few hundred to about 10,000 tokens without anyone asking for it. The model learns to reflect, verify and backtrack: “Wait, wait. Wait. That’s an aha moment.”
- **Went wrong:** R1-Zero was hard to read and mixed languages. DeepSeek-R1 therefore trains in four stages: cold-start SFT on a few thousand long chains; reasoning RL; rejection-sampling SFT on 800k samples; a final RL round that mixes checkers and preference rewards.
- **Tried and dropped** (their own “unsuccessful attempts”): process reward models (steps hard to define and label, reward hacking) and Monte Carlo tree search (a search space far larger than chess, a value model hard to train).
- **Distillation:** SFT of Qwen-32B on 800k R1 samples gives 72.6 percent on AIME; RL from scratch on the same base gives 47.0.



The post-training recipe of 2026

- 1. Pre-training, then mid-training** on math, code and long chains, so the base model is “RL-friendly”.
- 2. Cold-start SFT** on curated, often distilled, reasoning traces: fixes the format, seeds the behaviour. Quality over volume.
- 3. RL with verifiable rewards** on math, code, tool use and instruction checks, with a GRPO-family optimiser; increasingly several domain-specialist runs.
- 4. Preference and safety RL:** learned reward models, AI feedback (Constitutional AI, RLAIIF), rubric judges; DPO for smaller teams.
- 5. Distillation** into the shipped model and into small models; thinking budgets at inference. Post-training now takes more than a tenth of the pre-training compute (DeepSeek V3.2, 2025).

Stable: SFT and rejection sampling, RLHF with a KL leash, DPO, GRPO, verifiable rewards, best-of-N, distillation.

Contested: process reward models as a training signal, MCTS for text, rubric rewards, and whether RL adds capability or only sharpens it.



Which method when

- **Preference data and a small team:** SFT, then DPO. Iterate: sample from your own model, label, train again.
- **A checker exists** (math, code, structured output, retrieval with known answers): SFT cold start, then GRPO with verifiable rewards.
- **No checker, but a rubric:** a judge model with a checklist; keep a held-out judge and watch for gaming.
- **A small model for deployment:** distil from a stronger model's traces; cheaper than RL and it transfers behaviour the small model would not find on its own.
- **Behaviour, tone, safety:** RLHF or DPO with a KL leash and early stopping; measure sycophancy.
- **In every case:** track response length and entropy, and run a random-reward baseline before you believe a gain.

From Prompting to Reasoning

Recap of the prompting techniques of Lecture 11, then: how does a model learn to write its own chain of thought, and who tells it when the chain is good?

Recap: prompting techniques (Lecture 11)

- **Single path:** chain of thought (write the steps), program of thought (write code, let the interpreter compute).
- **Many paths, pick one:** self-consistency (vote), best-of-N with a verifier.
- **Search:** tree of thoughts, graph of thoughts, Monte Carlo tree search over steps.
- **Interaction:** ReAct, agents (Lecture 17).

All of these make the model think in a way the prompt prescribes, and all of them ask the model to judge its own steps.

The question for this lecture: can a model learn to produce such chains itself, and who tells it when a chain is good?

Reasoning?

- So, are CoT, ToT and SoT (and there are many more such ideas as Graph of Thought or Program of Thought) actually forms of “Reasoning”?
- How can we teach a model to create such more complex structures itself?

Reasoning

- Reasoning is a cognitive process that involves using evidence, arguments, and logic to arrive at conclusions or make judgments.
- Reasoning involves making inferences, evaluating arguments, and drawing logical conclusions based on available information. Different forms of reasoning are:
 - Deductive reasoning: All men are mortal (*major premise*). Socrates is a man (*minor premise*), therefore, Socrates is mortal (*conclusion*)
 - Inductive reasoning: uses logical process of induction: repeating observations lead to important generalizations “the sun always rises in the east”.
 - Abductive reasoning: is a type of logical thinking that involves forming the most likely explanation for a set of observations or evidence. It is often described as "inference to the best explanation," where conclusions are drawn based on the simplest and most plausible hypotheses, even if they are not definitively proven.
 - Formal reasoning: Formal reasoning is concerned only with the forms of arguments. Certain forms of arguments have been identified which are valid. In other words, if the original statements (or premises) in those arguments are true, then the conclusions must necessarily be true also.
 - Informal reasoning: Informal reasoning includes formal reasoning but it also is concerned with all the other elements of reasoning. It addresses the probability of truth of premises and conclusions. Informal reasoning is considered to be common, every-day reasoning.

Although “reasoning” is a term that is commonly used in literature and daily life, it is also an abstract concept that can refer to many things.

Reasoning with LLMs

- However, despite the strong performance of LLMs on certain reasoning tasks, it remains unclear whether LLMs are actually reasoning and to what extent they are capable of reasoning.
- Reasoning in LLMs is not deductive reasoning, inductive reasoning, abductive reasoning, or formal reasoning. It seems to be more like “informal reasoning” based upon small chunks of information obtained using CoT prompting driving a next-word prediction process.

Food for Thought

WSJ

Are LLMs actually the best methods to “reason”?

“A good example of this: An Atari 2600 running a program from 1979 can beat cutting-edge chatbots at chess. The bots tend to attempt illegal moves and quickly lose track of the positions of their pieces.

In essence, today’s transformer-based AIs are making predictions rather than reasoning logically. All this despite that they’ve been exposed through training to countless games and countless rulebooks. The Atari wins because it keeps the locations of pieces straight using an ancient and humble version of an internal world model: a database.”



https://www.wsj.com/tech/ai/world-models-ai-evolution-11275913?st=NLBCtV&reflink=article_email_share

So for now let's presume ...

- In the tradition of Alan Turing, that we are interested in the phenomena of reasoning without completely understanding how the human reasoning process actually works or can best be implemented in computer systems.
- That a chain of thought is the way to have LLMs solve more complex problems.
- That we can teach a model to write such chains itself: reward the chain (each step, or only its outcome) and let the model learn from that reward, as in Step 3 of the ChatGPT recipe.

Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Reward Models

- A **reward model (RM)** is a learned function $R_\phi(\cdot)$ that scores the quality of a model's output so we can:
 - rank candidate solutions,
 - give feedback for reinforcement learning (PPO, etc.),
 - or guide search (beam, MCTS, best-of-N).

How we define the reward depends on *what we care about*.

There are outcome-based reward models (scores the final outcome) and process-based reward models (scores all intermediate steps)

Outcome or process reward?

- **Outcome reward:** one scalar for the whole chain, based on whether the final answer matches the ground truth.
 - Every step gets the same credit, right or wrong; there is no signal about where the reasoning went wrong. The classic credit-assignment problem.
- **The obvious answer in 2023:** score each step (process supervision, next slides).
- **The surprise of 2025:** an outcome reward is enough when the model writes its reasoning out loud and many chains are sampled (DeepSeek-R1). The chain becomes the model's way to earn the reward, and it learns to check its own steps.
- **Process reward models remain useful** where they are cheap to apply: ranking candidate solutions at test time.

How can we Teach a LLM to create its own CoT prompts?

- Let's expand the idea of a process-supervised reward model (PRM) that scores each step of a CoT and then uses those scores to re-rank multiple sampled solutions (best-of-N).

Let

$$x^8 + 3x^4 - 4 = p_1(x)p_2(x) \cdots p_k(x),$$

where each non-constant polynomial $p_i(x)$ is monic with integer coefficients, and cannot be factored further over the integers. Compute $p_1(1) + p_2(1) + \cdots + p_k(1)$.

I notice that the given polynomial has even degree and only even powers of x , so I can try to make a substitution to simplify it.

Let $y = x^4$, then the polynomial becomes $y^2 + 3y - 4$, which is a quadratic equation.

I can factor this quadratic equation as $(y + 4)(y - 1)$, so the original polynomial is $(x^4 + 4)(x^4 - 1)$.

Now I need to factor each of these factors further, if possible, over the integers.

For the first factor, $x^4 + 4$, I recall the Sophie Germain identity, which says that $a^4 + 4b^4 = (a^2 + 2b^2 + 2ab)(a^2 + 2b^2 - 2ab)$.

Applying this identity with $a = x$ and $b = 1$, I get $x^4 + 4 = (x^2 + 2x + 2)(x^2 - 2x + 2)$.

For the second factor, $x^4 - 1$, I can use the difference of squares formula, which says that $a^2 - b^2 = (a + b)(a - b)$.

Applying this formula with $a = x^2$ and $b = 1$, I get $x^4 - 1 = (x^2 + 1)(x^2 - 1)$.

Then I can apply the difference of squares formula again with $a = x$ and $b = 1$, to get $x^2 - 1 = (x + 1)(x - 1)$.

So the original polynomial is $(x^4 + 4)(x^4 - 1) = (x^2 + 2x + 2)(x^2 - 2x + 2)(x^2 + 1)(x + 1)(x - 1)$.

These are all the irreducible factors over the integers, so $p_1(x) = x^2 + 2x + 2$, $p_2(x) = x^2 - 2x + 2$, $p_3(x) = x^2 + 1$, $p_4(x) = x + 1$, and $p_5(x) = x - 1$.

To find the sum of their values at $x = 1$, I just need to plug in $x = 1$ into each factor and add them up.

I get $p_1(1) = 1^2 + 2 \cdot 1 + 2 = 5$, $p_2(1) = 1^2 - 2 \cdot 1 + 2 = 1$, $p_3(1) = 1^2 + 1 = 2$, $p_4(1) = 1 + 1 = 2$, and $p_5(1) = 1 - 1 = 0$.

So the final answer is $5 + 1 + 2 + 2 + 0 = 10$.

Answer: 10

Let

$$x^8 + 3x^4 - 4 = p_1(x)p_2(x) \cdots p_k(x),$$

where each non-constant polynomial $p_i(x)$ is monic with integer coefficients, and cannot be factored further over the integers. Compute $p_1(1) + p_2(1) + \cdots + p_k(1)$.

This problem is about factoring a polynomial over the integers, which can be a tricky task.

One possible strategy is to look for rational roots of the polynomial, using the rational root theorem.

However, that would involve checking a lot of possible factors of -4 , and there is no guarantee that the polynomial has any rational roots at all.

Another possible strategy is to use some algebraic tricks to simplify the polynomial or reduce its degree.

I notice that the polynomial has only even powers of x , so I can try to substitute $y = x^2$ and see what happens.

Then the polynomial becomes $y^4 + 3y^2 - 4$, which is a quadratic equation in y^2 .

I can try to factor this quadratic equation by looking for two numbers that multiply to -4 and add to 3 .

Those numbers are 4 and -1 , so I can write $y^4 + 3y^2 - 4 = (y^2 + 4)(y^2 - 1)$.

Now I can substitute back $x^2 = y$ and get $x^8 + 3x^4 - 4 = (x^4 + 4)(x^4 - 1)$.

This looks promising, because I have reduced the degree of the factors from 8 to 4 .

I can try to factor each of these factors further, using the difference of squares formula.

I get $x^8 + 3x^4 - 4 = (x^2 + 2)(x^2 - 2)(x^2 + 1)(x^2 - 1)$.

I can apply the difference of squares formula again to the last factor and get $x^8 + 3x^4 - 4 = (x^2 + 2)(x^2 - 2)(x^2 + 1)(x + 1)(x - 1)$.

Now I have factored the polynomial completely into monic linear and quadratic factors with integer coefficients.

These are the $p_i(x)$'s that the problem is asking for.

To find the sum of their values at $x = 1$, I just need to plug in $x = 1$ into each factor and add them up.

I get $p_1(1) + p_2(1) + \cdots + p_k(1) = (1^2 + 2)(1^2 - 2)(1^2 + 1)(1 + 1)(1 - 1)$.

Simplifying, I get $p_1(1) + p_2(1) + \cdots + p_k(1) = (3)(-1)(2)(2)(0)$.

Multiplying, I get $p_1(1) + p_2(1) + \cdots + p_k(1) = 0$.

Answer: 0

Process vs. Outcome Supervision

- **Goal.** Compare process supervision (scoring each step in a solution) vs. outcome supervision (scoring only the final answer) by training reward models and testing which one picks better solutions.
- **Data.** They collect PRM800K: ~800k step-level labels on math CoTs from a large generator (GPT-4). Labelers mark each reasoning step as correct/incorrect/neutral.
- **CoT usage.** The generator is lightly tuned to output newline-delimited step-by-step solutions so steps can be scored individually; the PRM predicts the probability each step is correct and aggregates those into a single score per solution.
- **Selection at test time.** For each problem they sample N solutions, score them with the PRM, and pick the top-ranked one (“best-of-N”). No tree search; it’s reranking over a sampled set.
- **Result.** Process-supervised RM outperforms outcome-supervised RM on the MATH benchmark in best-of-N selection (they report a strong gain; e.g., ~78% solved in their large-scale setting).

Requires full annotation of each step of the process.

Train the PRM on each step-label.

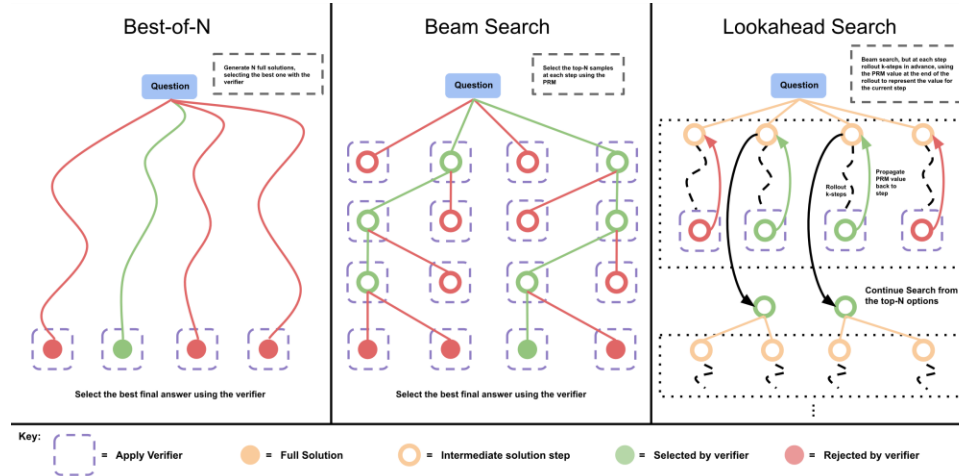
At inference time: generate many CoTs, rank using PRM, pick best.



Process reward models since 2023

- **Best-of-N with a PRM works:** Lightman et al. (2023) solve 78 percent of MATH test problems by picking the PRM's favourite out of 1,860 samples; the outcome-supervised model does worse at every N.
- **Automatic step labels:** Math-Shepherd (2023) estimates how good a step is from Monte Carlo completions: no human labels, 800k of them no longer needed.
- **As a training signal PRMs disappointed:** DeepSeek-R1 and Kimi k1.5 list them among the methods they dropped; Qwen (2025): Monte Carlo labels are noisy, and PRMs drift to scoring only the final step, which inflates best-of-N results.
- **Consensus 2026:** verifiable or outcome rewards for training; PRMs for ranking at inference, and only where the steps are well defined.

Three ways to spend compute at test time



- **Think longer:** one chain, more tokens (sequential; what o1 and R1 do).
- **Think several times and vote:** self-consistency, best-of- N with a verifier (parallel).
- **Search:** beam search or lookahead over steps, guided by a process reward model (both).
- **Snell et al. (2024):** allocate by difficulty. Compute-optimal allocation beats best-of- N by more than four times; on easy questions a small model with test-time compute beats a model fourteen times larger. On hard questions it does not.

Using Bayes again

- Self-consistency is very naturally phrased in Bayesian terms, and majority vote is basically the MAP solution of a simple Bayesian model.
- Reasoning paths are latent variables.
- You approximate the posterior over answers by Monte Carlo samples of these latent paths and then aggregate.
- Majority vote is the simplest possible marginalization rule; a Bayesian implementation would compute $P(\text{answer}=y|\text{samples})$ either:
 - choose the MAP answer (often = majority), or
 - keep the whole distribution for downstream calibration / uncertainty.

Thinking budgets and distillation

- **A reasoning model can be told how long to think:** a token budget (Claude, Gemini, Qwen3) or an effort level (GPT-5).
- **s1 (2025):** SFT on 1,000 distilled examples; when the model wants to stop, append “Wait” and it continues. Accuracy rises with the number of thinking tokens (figure).
- **Overthinking:** o1-style models spend thousands of tokens on “2 + 3” (Chen et al. 2024). Length penalties and “long to short” distillation are the standard fixes.
- **Distillation:** a small model trained on a reasoning model’s traces (R1-Distill, Sky-T1, s1) inherits the behaviour at a fraction of the cost of RL, and often beats RL on the same small model.

How many r in raspberry?

Question

Let's break down the process of counting the letter 'r' in the word "raspberry" ...

* First letter: 'r' - This is an 'r', count = 1.

* Second letter: 'a' - Not an 'r', count remains 1 ...

* Sixth letter: 'e' - Not an 'r', count remains 1.

* Seventh letter: 'r' - This is an 'r', count = 2.

* Eighth letter: 'y' - Not an 'r', count remains 2 ...

The number of 'r's in "raspberry" is 2.

Wait, let's re-read the question carefully. It asks "How many r in raspberry?" ... * r - a - s - p - b - e - r - r - y ... * First 'r' ... * Second 'r' ... * Third 'r' ... Count = 3 ...

Reasoning trace

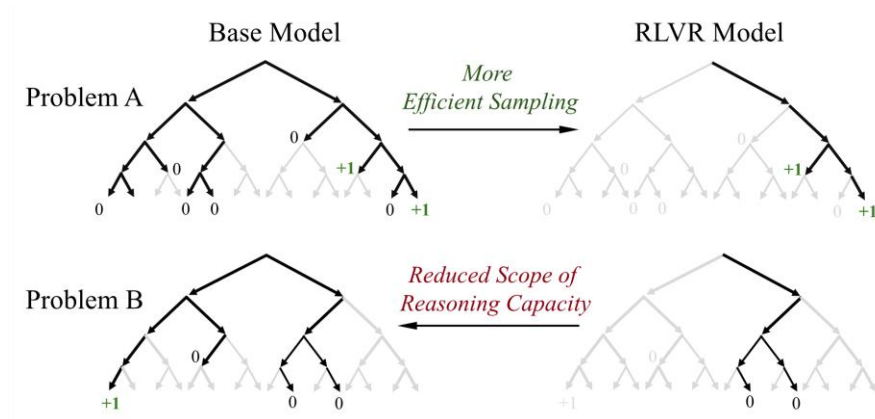
My initial answer of 2 was incorrect due to a quick reading of the word. **Final Answer:** The final answer is **3**

Response

Search or learning? MCTS versus RL for reasoning

- **Monte Carlo tree search:** treat reasoning as a tree; expand candidate steps, roll out to the end, back up a value, follow the best branch. Needs a state you can reset and a value you can estimate.
- **DeepSeek tried it for text and dropped it:** the token-level search space is far larger than a chess board, and a fine-grained value model was hard to train.
- **RL on the outcome won for text:** the model learns to explore and backtrack inside one linear chain, at the cost of one sample.
- **Where search still wins:** formal mathematics (AlphaProof searches Lean proofs; silver at the IMO 2024), and as a data generator (rStar-Math: MCTS with code-verified steps brings a 7B model to the level of o1-preview). More in the lecture on agents.

Does RL teach new reasoning, or sharpen it?

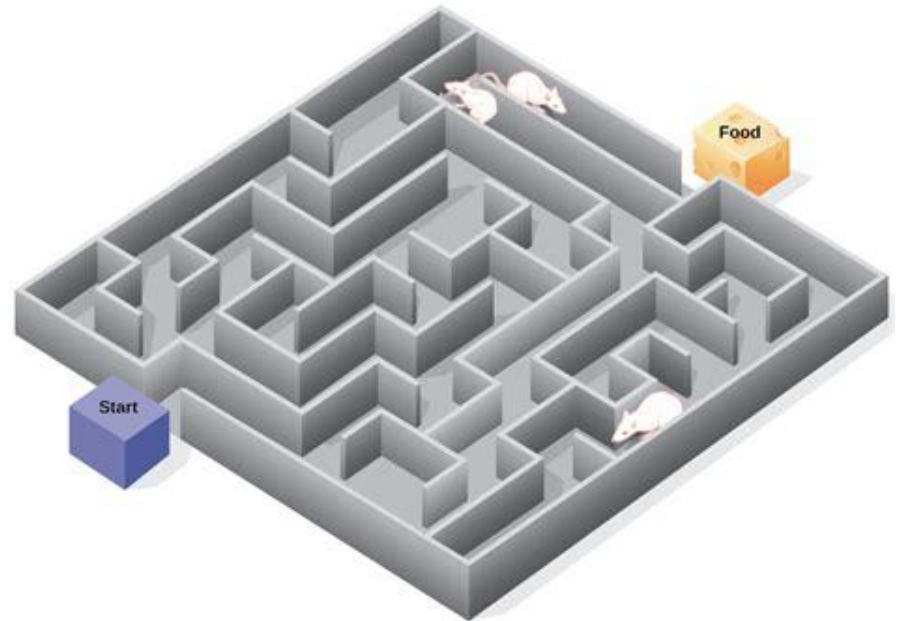


- **pass@k:** the chance that at least one of k samples is correct. The figure: RL concentrates the base model's probability mass on paths that already led to the answer.
- **Yue et al. (2025):** the RL model wins at small k, the base model catches up or wins at large k. Short RL runs re-weight; they rarely find a path the base never had.
- **Spurious rewards (2025):** on Qwen2.5-Math a random reward gives +21 points on MATH-500, the true reward +29; the gain amplified a habit the base already had. Nothing of the kind on Llama.
- **Against:** prolonged RL with KL control and reference resets (ProRL, 2025) beats the base at every k, including problems the base never solved.
- **Honest reading:** short runs sharpen, long diverse runs expand a little, distillation injects new patterns. Always run the random-reward control.

From Stochastic Parrots to Maze Rats ...



Emily M. Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. 2021. On the Dangers of Stochastic Parrots: Can Language Models Be Too Big? 🦜 In Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FACCT '21). Association for Computing Machinery, New York, NY, USA, 610–623.
<https://doi.org/10.1145/3442188.3445922>



From the paper: Let's Verify Step by Step. Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, Karl Cobbe, 2023.
<https://arxiv.org/abs/2305.20050>

“OpenAI plans to build data centers that would each require the power generated by approximately five large nuclear reactors, enough for almost 3 million homes”

“This is back to a million monkeys typing for a million years generating the works of Shakespeare”

TECHNOLOGY

THE GPT ERA IS ALREADY ENDING

Something has shifted at OpenAI.

By Matteo Wong

“There are now two axes on which we can scale: training time and run time, monkeys and years, parrots and rats”

Can we trust the chain of thought?

- **The chain is text the model produced**, not a log of its computation. Anthropic (2025): given a hint that changes the answer, Claude 3.7 mentions the hint in 25 percent of its chains, DeepSeek-R1 in 39 percent.
- **Models trained to exploit a reward hack** verbalised the hack in fewer than 2 percent of cases.
- **Yet the chain is our best window**: when a task is hard, the reasoning has to pass through legible tokens. Researchers from OpenAI, DeepMind and Anthropic call this “a new and fragile opportunity” (Korbak et al. 2025): optimise the chain for style or length and the window closes.
- **Compare Lecture 12**: attention is not an explanation, and neither is a chain of thought.

References: alignment and reasoning

- Schulman et al. 2017, Proximal Policy Optimization Algorithms, arXiv:1707.06347
- Ouyang et al. 2022, Training language models to follow instructions with human feedback, arXiv:2203.02155
- Zelikman et al. 2022, STaR: Bootstrapping Reasoning with Reasoning, arXiv:2203.14465
- Gao, Schulman, Hilton 2022, Scaling Laws for Reward Model Overoptimization, arXiv:2210.10760
- Bai et al. 2022, Constitutional AI: Harmlessness from AI Feedback, arXiv:2212.08073
- Rafailov et al. 2023, Direct Preference Optimization, arXiv:2305.18290
- Lightman et al. 2023, Let's Verify Step by Step, arXiv:2305.20050
- Ahmadian et al. 2024, Back to Basics: REINFORCE-style optimization for RLHF, arXiv:2402.14740
- Shao et al. 2024, DeepSeekMath (GRPO), arXiv:2402.03300
- Grattafiori et al. 2024, The Llama 3 Herd of Models, arXiv:2407.21783
- Snell et al. 2024, Scaling LLM Test-Time Compute Optimally, arXiv:2408.03314
- OpenAI, September 2024, Learning to reason with LLMs (o1), openai.com/index/learning-to-reason-with-llms
- Lambert et al. 2024, Tulu 3 (RL with verifiable rewards), arXiv:2411.15124
- DeepSeek-AI 2025, DeepSeek-R1, arXiv:2501.12948 and Nature 645 (2025)
- Muennighoff et al. 2025, s1: Simple test-time scaling, arXiv:2501.19393
- Yu et al. 2025, DAPO, arXiv:2503.14476; Liu et al. 2025, Dr. GRPO, arXiv:2503.20783
- Yue et al. 2025, Does RL Really Incentivize Reasoning Capacity Beyond the Base Model?, arXiv:2504.13837
- Chen et al. 2025, Reasoning Models Don't Always Say What They Think, arXiv:2505.05410
- Shao et al. 2025, Spurious Rewards, arXiv:2506.10947; Korbak et al. 2025, CoT Monitorability, arXiv:2507.11473
- Khatri et al. 2025, The Art of Scaling Reinforcement Learning Compute for LLMs, arXiv:2510.13786



From LLMs to Agents

We will discuss this in the Lecture on Agents.

Agent = LLM (“brain in a jar”) + memory, tools, planning & goals.



Maastricht University

Questions?



Maastricht University

Exercise 1: Pre-Training Parameter Budget and Compute Scheduling (BERT-Base)

Review the parameter count derivation and pre-training regime for BERT-Base ($L = 12$ layers, hidden dimension $d = 768$, $h = 12$ attention heads, vocabulary size $V = 30,000$, and maximum sequence length $P = 512$).

- Calculate the parameter counts for the embedding layer components: Token Embeddings ($V \times d$), Positional Embeddings ($P \times d$), and Segment Embeddings ($2 \times d$). Explain why all three components must remain fully trainable rather than freezing the token embeddings.
- For a single Transformer layer, show how the parameter count is calculated across:
 - Multi-Head Self-Attention: Query, Key, Value projections ($3 \times d^2$) and the output projection (d^2).
 - Intermediate Feed-Forward Network: the expansion to $d_{\text{ff}} = 3072$ ($d \times 3072$) and projection back ($3072 \times d$).
 - LayerNorm parameters.
 - Combine these to verify that each layer contains approximately 7.08M weights, resulting in ~ 108.4 M total trainable parameters across 12 layers.
- Using the pre-training schedule from the lecture:

$$\text{Epochs} = \frac{\text{Steps} \times \text{Batch Size} \times \text{Sequence Length}}{\text{Total Tokens in Corpus}}$$

- Given a corpus of 3.63 billion tokens, a batch size of 256 sequences, and a sequence length of 512 tokens, calculate the number of steps required to complete 1 epoch and verify why 1,000,000 steps equates to approximately 40 epochs. Why are only 5–10 epochs recommended when performing domain-specific continual pre-training (e.g., BioBERT or LegalBERT)?

Exercise 2: Alignment, from PPO to DPO, GRPO and verifiable rewards

The lecture follows alignment from the three-stage recipe of InstructGPT (SFT, reward model, PPO) to the critic-free methods of 2024 and 2025.

- Step 1 (SFT): why was the checkpoint selected by reward-model score rather than by validation loss, although the validation loss started to rise after one epoch?
- Step 3 (PPO): how is the environment a bandit, what does the per-token KL penalty prevent, and why does PPO need four networks in memory?
- DPO: starting from the KL-regularised objective, explain in words how the reward model is eliminated. What does DPO lose compared with on-policy RL, and how does iterative DPO repair it?
- GRPO: what replaces the value model, how is the advantage of one sample computed, and why is this called grading on a curve? Name two of the 2025 fixes and the bias each repairs.
- RLVR: when can a checker replace the reward model, and what does that buy against reward hacking?

Exercise 3: Test-Time Reasoning, Reward Supervision and Verifiable Rewards

The lecture moves from prompting (Lecture 11) to models that learn to write their own chain of thought.

- Self-consistency as a Bayesian estimate: why is majority voting over sampled chains the maximum a posteriori answer, and what does it assume about the errors of individual chains?
- Explain the credit-assignment problem of an outcome reward, how a process reward model (Lightman et al. 2023) addresses it, and how best-of-N uses such a model at test time.
- DeepSeek-R1-Zero was trained with an outcome reward only. What emerged during training, what went wrong, and why did the authors drop process reward models and tree search?
- pass@k: an RL-trained model beats its base model at $k = 1$ but not at $k = 256$. Give the two readings of this result and the control experiment that separates them.
- Name the three ways to spend compute at test time and state when, according to Snell et al. (2024), a small model with more thinking beats a larger model.

Discussion



Maastricht University

On Process Supervision: Can a Model Learn to Reason Without Us Grading Every Step?

Process supervision scores every step of a solution; outcome supervision scores only the answer. Process supervision works better, and PRM800K needed 800,000 human labels on individual reasoning steps to show it.

If a model only reasons well where humans have graded the reasoning, has it learned to reason, or have we moved human supervision one level deeper into the model's thinking? Is there a point at which supervising the process is just doing the thinking ourselves?

Where does step-level supervision stop scaling, what can reinforcement learning with only verifiable outcomes offer in its place, and what is lost when the only thing that can be rewarded is being right at the end?

On Parrots and Maze Rats: Does Searching Over Guesses Amount to Thinking?

“There are now two axes on which we can scale: training time and run time, monkeys and years, parrots and rats.” For a decade, scaling meant bigger models on more text. Now a model is given time to think: sample many chains, search a tree, score the steps, pick the best.

The engine inside is still a next-token predictor. Does wrapping it in search and voting turn prediction into deduction, or is a rat in a maze still guessing, only more systematically?

Human reasoning also looks like search over proposals with a critic attached. If the architecture is the same, what, if anything, is the difference? And if there is none, why does it still feel like one?



On the Brain in a Jar: Does Attaching a World Give You a World Model?

A bare language model is a brain in a jar: text in, text out, no world, no body, no goal. An agent is that jar with things attached: a memory, tools, a planning loop. A 1979 chess program on an Atari still beats it, because the Atari keeps an exact board in memory and the model keeps a guess.

If the engine in the middle has no world model, does attaching a world (databases, APIs, sensors) give it one, or does it only give its guesses longer arms?

Is grounding something a system has, or something a system does? And if an agent acts in the world reliably without an internal model of it, does it matter that the model is missing?